ScienceDirect

Real World Multicore Embedded Systems
2013, Pages 481-515

Chapter 14 - Multicore Synchronization Hardware

Jim Holt

Show more

Outline | Share Cite

https://doi-org.byui.idm.oclc.org/10.1016/B978-0-12-416018-7.00014-6 ↗
Get rights and content ↗

Full text access

Synchronization hardware is a fundamental requirement for concurrent software. Ultimately, software depends upon strong guarantees for atomicity, which can only be provided by hardware. Fortunately, hardware can provide a simple but powerful programming interface to higher-level software using only a few synchronization primitives. These few primitives can support a surprisingly broad range of capabilities in software. Yet, even with hardware support to ensure functional correctness, there is an unavoidable performance overhead to pay for synchronization. In recent years, hardware designed to replace precise locking with speculative lock avoidance (e.g., *lock-free programming*) has begun to emerge. This chapter provides an in-depth look at the lowest level hardware/software interface for synchronization, along with explanations of how the underlying hardware ensures atomicity, and considerations related to weakly consistent memory models. This is complemented with a discussion of various lock avoidance techniques.

Previous Next

Keywords

hardware synchronization; memory consistency models; lock-free programming

Chapter Outline

Chapter overview 481

Instruction set support for synchronization 483

Test-and-set 484

Compare-and-swap 487

Load-reserved/store-conditional 489

Creating new primitives 491

Compiler intrinsics 493

Hardware support for synchronization 494

Bus locking 494

Load-reserved and store-conditional 495

Hardware support for lock-free programming 496

Lock-free synchronization with hardware queues 497

Decorated storage operations 499

Messaging 500

Inter-device messaging with hardware queues 501

Messaging with DMA 502

Inter-processor interrupt facilities 504

Messaging in non-coherent systems 505

Hardware transactional memory 507

Memory subsystem considerations 509

Memory ordering rules 510

Using memory barriers and synchronizations 512

Conclusions 514

References 514

Chapter overview

Synchronization is an essential capability for programming multicore systems. This requirement spans operating systems, virtualization software, device drivers, system libraries, run-times, and application software. However, despite the existence of software-only solutions such as Peterson's algorithm [1], it has been shown to be impractical to achieve these capabilities in multicore systems with software alone. Thus, it is essential for multicore systems to provide hardware support in order to ensure correctness and efficiency for synchronization in concurrent software. This chapter provides a detailed look at the hardware/software interface for synchronization and presents underlying hardware support commonly found in multicore systems.

Programmers find themselves in need of two basic forms of synchronization: *data synchronization*, and *control synchronization*. Data synchronization prevents race conditions and corruption by coordinating concurrent access to shared data. A typical mechanism for data synchronization is some form of *lock*. A process must obtain a lock before accessing shared data, and then release the lock when finished. Control synchronization coordinates parallel algorithm execution to ensure correctness. A typical mechanism for control synchronization is a *barrier*. A process or thread enters the barrier, waits for the other processes or threads, and eventually all processes or threads leave the barrier together. Despite these apparent differences, for most current

multicore systems, the lowest level of software implementation for both types of synchronization will generally depend upon a combination of locking and shared data to provide the higher-level synchronization semantics.

Several decades of research and development have been devoted to providing hardware synchronization mechanisms. Today, hardware support for synchronization is ubiquitous, spanning from massively parallel supercomputers to deeply embedded multicore systems. Most modern computer instruction sets include operations for synchronization, and all processors adhering to these instruction sets provide underlying hardware to ensure correct and atomic operation. (It is not always necessary to have strictly atomic operations in a multicore system; sometimes it is sufficient to guarantee that an operation appears atomic from the point of view of software. Many of the mechanisms discussed in this chapter are *speculatively atomic*, e.g., allowing software to detect when atomicity has been violated. In these cases it is up to software to recover from atomicity violation.) Many different idioms for atomic synchronization operations have been invented, but in recent years three primitive operation classes have emerged as the mainstream: *test-and-set*, *compare-and-swap*, and *load-reserved/store-conditional*.

A number of additional synchronization operations can be defined using these few instruction set primitives, and compilers also provide a variety of intrinsic operations built upon them. The section “Instruction set support for synchronization” examines each of the three most common primitives, provides examples of how to create additional synchronization operations using them, and illustrates the use of compiler intrinsics.

Sometimes it is desirable to avoid synchronization. This may seem counterintuitive, and, while synchronization is an effective and time-proven approach, using locks and shared memory can be quite inefficient when there is significant contention for shared data. This is unavoidable as the number of processes or threads increases and as the number of available cores increases. Therefore, many multicore systems provide additional mechanisms to avoid synchronization altogether. These *lock-free* techniques can yield higher performance and better power efficiency in certain situations. The “Hardware support for lock-free programming” section will examine a number of these approaches for both data and control synchronization and discuss their applicability.

Instruction set support for synchronization

Low-level hardware synchronization mechanisms are typically available to programmers via function calls in system libraries. These functions encapsulate architecture-specific instructions that rely on underlying hardware mechanisms to provide atomicity. The fundamental capability required for synchronization is the ability to read/modify a shared memory location atomically. Processor architectures may provide this ability as true blocking operations, or, alternatively, by splitting the read and modify transactions while providing speculative atomicity. Whichever approach is taken, the goal is to guarantee that only one thread or process is active in a critical section of code at a time – thereby avoiding race conditions and deadlocks.

Most modern processors provide instructions that implement one or more forms of a few common idioms. These idioms include blocking atomic approaches such as *test-and-set* or *compare-and-swap*, and speculatively atomic operation pairs such as *load-reserved/store-conditional*. Regardless of which semantics an architecture provides, the underlying principle is the ability to atomically read/update a memory location. In the strictest sense, atomicity means that nothing will intervene between reading the value from a specific memory location and updating the value stored at the same memory location. In practice, there are a few ways to achieve this in hardware, but we will defer that discussion until the “Hardware support for synchronization” section.

When implementing synchronization routines from instruction set primitives, it is important to understand that the primitives themselves are not necessarily atomic in the strictest sense. In many cases the primitive operations only provide software the means to request atomicity for a causally related pair of operations (one for the memory read operation and one for the memory update operation) and a mechanism to detect when atomicity has been violated. Therefore, when programming routines using speculatively atomic operation pairs, a programmer must be cognizant of how to detect and recover from atomicity violations.

Given the complexities of managing speculative atomicity, the benefit of wrapping instruction-level primitives in library routines is twofold. First, it is clearly difficult to build and test a synchronization library. Once this has been done successfully, the result should be reused as much as possible! Second, encapsulating processor-specific instructions within a high-level library interface provides for better application portability. Portability is further aided by the fact that many variations on high-level synchronization routines can be built from a few low-level primitives. Thus, even when a processor does not directly provide certain operations in its instruction set, it is still possible to implement equivalent functionality in library routines.

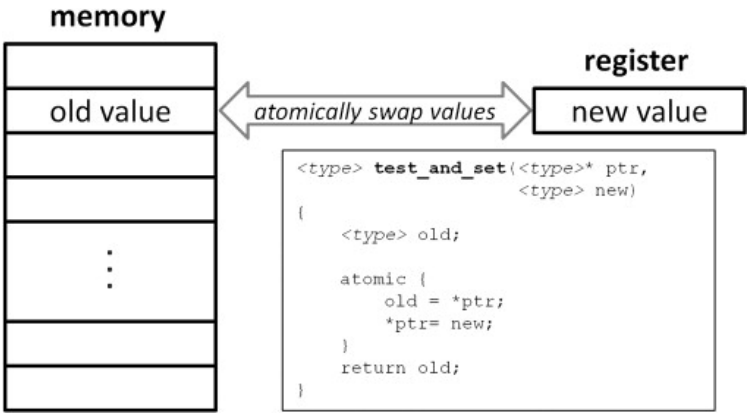
Test-and-set

The test-and-set operation modifies the contents of a memory location and returns the old value from that location in a single atomic operation. This operation is called “test-and-set” because it allows the programmer to atomically swap values between a register and memory (*set*) and then to evaluate the data retrieved from memory to make a programming decision (*test*). This will become clearer with an example of using test-and-set to create mutexes.

It is easy to imagine how this can be used to obtain a lock. For example, we can define a memory location representing a lock to have the value 1 if locked and the value 0 if unlocked. The SPARC architecture provides these semantics via the `ldstub` instruction [2], while the Intel architecture provides them via the `bts` instruction [3].

There are certainly subtle differences between instructions that provide test-and-set semantics. For example, the `bts` instruction operates on a byte, while the `ldstub` instruction operates on an unsigned byte. Hence, it’s imperative to use correct datatypes when wrapping assembly language within another language such as C, and furthermore it is highly desirable to hide these differences from higher-level programming constructs beneath an abstraction layer.

Setting low-level differences aside, the generalized semantics of test-and-set are illustrated by the psuedo-code in Figure 14.1. This operation is called “test-and-set”, because it allows the programmer to atomically swap values between a register and memory (*set*), and then to evaluate the data retrieved from memory to make a programming decision (*test*). This will become clearer with an example of using test-and-set to create mutexes.



Download: Download full-size image

Figure 14.1. Pseudo-code for test-and-set operation.

The implementation of a software `test_and_set()` routine itself is straightforward in assembly language; however, as discussed above, the specifics will vary by architecture. This is illustrated in Figure 14.2 using the SPARC `ldstub` instruction [4]. Note the extra steps taken to ensure that the 32-bit register is zero-filled since `ldstub` operates on unsigned byte data.

```
test_and_set:
        ldstub          [%o0], %o0          ! atomic load + set
        sll             %o0,24, %o0         ! zero fill
        retl                                ! result register
        srl             %o0,24, %o0         ! return
```

Download: Download full-size image

Figure 14.2. Assembly code for test_and_set() operation using SPARC `ldstub` instruction.

Given an atomic test-and-set primitive, it is straightforward to implement high-level language routines for locking and unlocking a mutex. This is depicted by the C code in Figure 14.3. A mutex is first initialized to zero. Then, the mutex can be locked by calling `lock_mutex()`. The implementation of `lock_mutex()` will remain in the `while` loop as long as another process has the mutex locked (e.g., as long as the return value of calling `test_and_set()` is one). It will return successfully once the lock is obtained (e.g., when the return value of calling `test_and_set()` is zero, indicating no other process has the lock).

```
typedef struct { int val; } mutex_t;

void init_mutex(mutex_t* mutex) { mutex->val = 0; }

void lock_mutex(mutex_t* mutex) {
    while(test_and_set(&mutex->val,1) == 1); /* wait */
}

void unlock_mutex(mutex_t* mutex) { mutex->val = 0; }
```

Download: Download full-size image

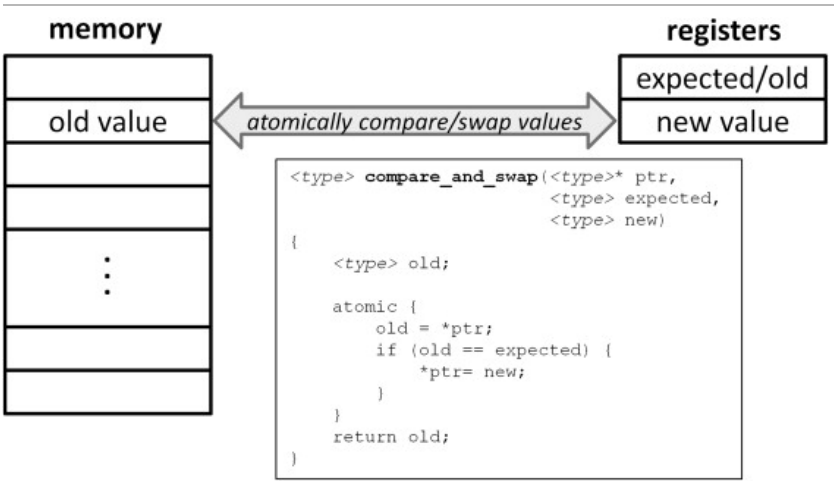
Figure 14.3. Simple C code for mutex with test_and_set() operation.

You can see that this simple mutex interface provides a suitable abstraction layer that can hide the differences between a `bts` or `ldstub` implementation from the programmer. However, a production-quality implementation would need to take other considerations into account, such as how to minimize wasted processor cycles in the `while` loop (since the process is effectively blocked while waiting to obtain the lock) or disabling interrupts in situations where there is the possibility of deadlocks (for example, in operating system spin-locks).

Revisiting the code fragment implementing the test-and-set operation (Figure 14.2), it is important to note that, in a multicore system, it will sometimes be necessary to insert a memory barrier or an instruction to lock the bus when executing an "atomic" instruction. This depends on the semantics of the particular machine and will be discussed in more detail in "Hardware support for synchronization" below. Regardless of the finer machine-specific details, the code that manages these semantics should be contained within the library routines implementing the locking primitives, thus ensuring correct operation for all users of the library.

Compare-and-swap

The compare-and-swap operation atomically compares the contents of a memory location with a user-specified value contained in a processor register and – *if they are equivalent* – modifies the contents of that memory location to a new user-specified value contained in another register (Figure 14.4). This operation is available in the SPARC architecture via the `cas` instruction, and in the Intel x86 architecture via the `cmpxchg` instruction. When using compare-and-swap, it is up to the programmer to test whether the swap actually occurred, and to wait or retry as necessary.



Download: Download full-size image

Figure 14.4. Pseudocode for compare-and-swap operation.

As with the test-and-set primitive, it is straightforward to provide a high-level programming interface to the low-level compare-and-swap operation. Encapsulation of compare-and-swap within a C library routine is illustrated for the Intel `cmpxchg` instruction in Figure 14.5. Note that the `cmpxchg` instruction requires the expected value to already be in the processor's

accumulator register (EAX). If the value at the destination location in memory is equal to the value in the EAX register then the destination location is updated with the new value. Otherwise, the EAX register is loaded with the original value of the destination location. The *lock* prefix shown in Figure 14.5 is used to explicitly lock a shared bus, as will be discussed later in more detail.

```
unsigned char compare_and_swap(int *ptr, int old, int new)
{
    unsigned char ret;

    /* Note that sete sets a 'byte' not the word */

    __asm__ __volatile__ (
            "  lock\n"
            "  cmpxchgl %2,%1\n"
            "  sete %0\n"
            : "=q" (ret), "=m" (*ptr)
            : "r" (new), "m" (*ptr), "a" (old)
            : "memory");

    return ret;
}
```

Download: Download full-size image

Figure 14.5. Inline assembly code for compare_and_swap() operation using Intel `cmpxchgl` instruction.

Given the existence of a simple mutex library such as the one used for the code in Figure 14.3, only a small modification is required in order to use compare-and-swap instead of test-and-set (Figure 14.6). This example illustrates the portability benefits of a software abstraction layer used to hide the details of the underlying hardware semantics.

```
void lock_mutex(mutex_t* mutex) {

    while(compare_and_swap(&mutex->val, 0, 1) == 1); /* spin */

}
```

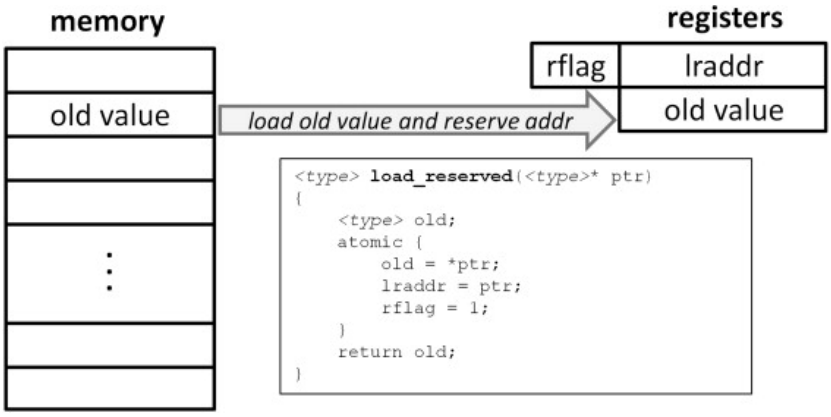
Download: Download full-size image

Figure 14.6. Modified C code for mutex lock with compare_and_swap() operation.

Again, as with test-and-set, use of compare-and-swap will likely involve other machine-specific considerations, especially for multicore systems, as will be detailed in “Hardware support for synchronization” below.

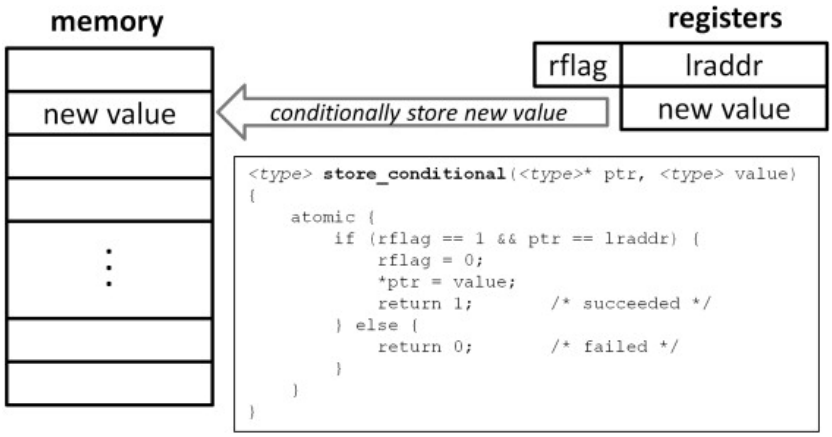
Load-reserved/store-conditional

For multicore systems, there will be performance implications when using atomic operations like test-and-set or compare-and-swap. The reasons will become clear from the discussion of underlying hardware support later. Recognizing this scalability concern, many modern architectures provide a speculative means to read/update memory via separate load-reserved and store-conditional operations. These operations depend on the system coherence mechanism to determine when an operation that occurs between the load-reserved and the store-conditional breaks the logical atomicity of the pair. A system which implements this paradigm will provide a means for software to detect an atomicity violation so that the operation pair can be retried. The two operations are depicted in Figure 14.7 and Figure 14.8. Architectures that provide these operations include Power Architecture® (`lwarx/stwcx` instructions [5]), ARM (`ldrex/strex` instructions), MIPS (`ll/sc` instructions [6]), and Alpha (`ldl_l/stl_c` instructions [7]).



Download: Download full-size image

Figure 14.7. Pseudo-code for load-reserved operation.



Download: Download full-size image

Figure 14.8. Psuedo-code for the store-conditional operation.

When a load-reserved operation is executed, the hardware will save the reserved address and will set a flag to indicate its reservation. When a subsequent store-conditional operation is executed, the flag and address will be compared and, if there is a match with the most recent load-reserved, then the store can proceed. However, whenever the system coherency mechanism sees a store transaction to the reserved address, the reserved flag will be cleared. If this were to occur after the load-reserved operation but before the store-conditional operation then the store-conditional would fail.

Implementing a mutex lock with load-reserved and store-conditional is illustrated in Figure 14.9. The power of separating the load and update operations into a pair of speculative operations is evidenced by the ability to insert additional code between the operations. This ability allows one to create additional atomic primitives with ease (see below), and provides opportunities to better manage overall system performance and efficiency.

```
void lock_mutex(mutex_t* mutex) {
    while(1) {
        while(load_reserved(&mutex->val) == 1) ; /* wait */
        if (store_conditional(&mutex->val,1) == 1)
           return;  /* locked */
    }
}
```

Download: Download full-size image

Figure 14.9. Modified C code for mutex lock with load_reserved() and store_conditional() operations.

In the following sections we will see how to build additional atomic primitives using the load-reserved/store-conditional operations. This is followed by a brief explanation of how compilers provide many of these operations as intrinsics and discussion of the pros and cons of using intrinsics for synchronization.

Creating new primitives

It is straightforward to create additional "atomic" primitives using load-reserved and store-conditional. The examples in this section will illustrate this using the Power Architecture® instructions `lwarx` (*load word and reserve*) and `stwcx` (*store word conditional*) [5]. The functional equivalent of the test-and-set operation defined earlier is illustrated in Figure 14.10. Of course this code could be quite inefficient if there are many processes or threads contending for the mutex lock. In this case the library implementer would probably add additional code to interact with the operating system's scheduler to cause the calling process to block for a short period of time before retrying rather than greedily consuming processor cycles merely busy-waiting. Note that the example code in Figure 14.10 does not take into account the memory semantics of Power Architecture. This would require a memory barrier following the `stwcx` instruction to guarantee correct operation. This will be elaborated upon in "Using memory barriers and synchronizations" below.

```
loop:     lwarx   r4, 0, r3     # load and reserve
          stwcx.  r5, 0, r3     # store new value if still reserved
          bc      4, 2, loop    # loop if lost reservation
```

Download: Download full-size image

Figure 14.10. Assembly code for test_and_set() operation using load-reserved/store-conditional.

It is also possible to implement compare-and-swap semantics using load-reserved/store-conditional operations. This is depicted in Figure 14.11. This example implements exactly the original IBM System/370 style of compare-and-swap.

```
loop:     lwarx   r6, 0, r3     # load and reserve
          cmpw    r4, r6        # first 2 operands equal?
          bc      4, 2, exit    # exit if not
          stwcx.  r5, 0, r3     # store new value if still reserved
          bc      4, 2, loop    # loop if lost reservation
exit:     or      r4, r6, r6    # return value from memory
```

Download: Download full-size image

Figure 14.11. Assembly code for compare_and_swap() using load-reserved/store-conditional.

Other potentially useful atomic operations are *fetch-and-add* and *fetch-and-and*, as well as all their arithmetic and logical cousins such as *subtract*, *multiply*, *or*, *xor*, etc. Two of these are illustrated in Figure 14.12. It is left to the reader to expand these into other variations.

```
fetch-and-add
loop:      lwarx   r5, 0, r3     # load and reserve
           add     r0, r4, r5    # ADD r4 to r5, place in r0
           stwcx.  r0, 0, r3     # store r0 if still reserved
           bc      4, 2, loop    # loop if lost reservation

fetch-and-and
loop:      lwarx   r5, 0, r3     # load and reserve
           and     r0, r4, r5    # AND r4 with r5, place in r0
           stwcx.  r0, 0, r3     # store r0 if still reserved
           bc      4, 2, loop    # loop if lost reservation
```

Download: Download full-size image

Figure 14.12. Fetch-and-add and fetch-and-and.

Compiler intrinsics

Compilers such as gcc [8] also offer programmers access to basic synchronization primitives via intrinsics. Intrinsic functions, sometimes referred to as built-ins, are functions which the compiler has special knowledge of and whose implementation can thus be optimized intelligently by the compiler. Intrinsics can therefore be thought of as similar to inline functions with a great likelihood of optimal performance.

Because compilers such as gcc support multiple target processors, using intrinsics can lead to portable code. However, one should be aware that synchronization primitives provided by compiler intrinsics may not have the same performance properties as synchronization primitives in system libraries since an intrinsic may not cooperate with the kernel scheduler in the same way that a run-time library would. For example, if the `lock_mutex()` routine were to be implemented using a compare-and-swap routine from a system library, the operating system has the ability to block the calling process on an unsuccessful compare-and-swap call. The process would eventually be rescheduled and hopefully could obtain the lock at that time. An instrinsic-based version would have to busy-wait in the lock_mutex() routine shown in Figure 14.13, potentially wasting considerable cpu cycles.

```
gcc intrinsic
bool __sync_bool_compare_and_swap (type *ptr, type oldval, type newval)


void lock_mutex(mutex_t* mutex) {
    while(__sync_bool_compare_and_swap(&mutex->val, 0, 1) == false); /* spin */
}
```

Download: Download full-size image

Figure 14.13. Mutex lock routine using gcc intrinsic.

Hardware support for synchronization

An instruction set defines the hardware/software interface for synchronization operations. Underlying hardware must efficiently and correctly provide the semantics guaranteed by this interface. A number of techniques have been developed, and these techniques have varying trade-offs between ease of implementation (e.g., complexity) and performance. In general, as multicore systems continue to provide increasingly more cores on a single die, hardware support for synchronization will take various steps to mitigate both complexity and performance issues. This section explores these various mechanisms and trade-offs.

Bus locking

Single-processor systems can guarantee that atomic operations such as compare-and-swap will in fact be performed atomically (since there are obviously no other processors attempting to utilize the interconnect and memory at any given time). The easiest

way to ensure atomicity of such operations in a shared-bus multicore system is to lock the bus for the duration of the transaction. Some architectures provide automatic bus locking in the hardware for certain atomic instructions (for example the Intel `xchg` instruction). Some architectures also provide an explicit means for the programmer to lock the bus from software. In Intel architectures, this is done by specifying the *lock* prefix to atomic operations such as `cmpxchg` (see "Compare-and-swap", and Figure 14.5).

In multicore systems with explicit software locking, the programmer must beware: an atomic instruction such as `cmpxchg` must be combined with the lock prefix to guarantee the operation will be performed atomically! The micro-architectural details can be subtle, and semantics will vary depending on the processor implementation. For example, older Intel processors always assert the bus lock signal during execution of an instruction with the lock prefix, even when the data to be operated on is in the processor cache. However, in newer Intel processors, when the data to be operated on is in the cache and the cache is configured as write-back, then the processor implementation may choose not to assert the lock signal on the bus. In this situation, the processor may instead modify the memory location internally to its cache and then allow the cache coherency mechanism to provide underlying atomicity. Clearly these subtle but important differences can lead to quite different performance characteristics in synchronization code.

Load-reserved and store-conditional

In a multicore system that implements bus locking, the bus must remain locked for the duration of an atomic read/modify operation. This can be a performance bottleneck even for simple buses with few cores, and it can be quite a significant issue in split-transaction buses with many cores. This has motivated modern architectures to provide split operations such as load-reserved and store-conditional. The mechanisms for managing and monitoring the reservation of a load-reserved/store-conditional operation pair can vary considerably in hardware. Regardless of how it is implemented in the hardware, the essential requirement is to guarantee that a reservation for a store-conditional must still be valid for the store to succeed. For this to work correctly, the hardware must maintain a reservation context and track accesses to the target memory address.

The MIPS R4000 architecture includes the `LLAddr` register and the user-transparent `LLBit` for this purpose [6]. When an `ll` instruction is executed, the target address will be retained in the `LLAddr` register and the `LLBit` will be set (refer to Figure 14.7, Figure 14.8). A subsequent `sc` instruction execution will cause the hardware to check that the store address matches the contents of the `LLAddr` and that the `LLBit` is still set. The target register will be loaded with the value 1 in the case of a successful store or 0 otherwise. In general, the `LLBit` will be cleared when any local or system coherency event occurs that indicates a potential change to the data at the target address. This sequence of events is complicated by the fact that a processor may not have the target address cached at the time of the store-conditional operation (for example, the cache line in question may have been replaced after the `ll` instruction executed but before the `sc` instruction executes). Therefore, coherency protocols must be carefully designed and verified such that the reserved address and linked bits are properly maintained. In some coherency implementations (for example directory-based protocols), this responsibility may be assigned to an external hardware agent. In this case, the processor will be required to communicate additional information to the external agent to ensure that tracking of the `LLBit` can be delegated outside the processor. Interactions between the processor core and an external agent are transparent to software and are handled automatically by the hardware.

Hardware support for lock-free programming

The act of waiting for a lock can be either *blocking* or *busy-waiting*. Blocking is not typically provided by hardware but, instead, must be coordinated in software between the locking routine and the operating system scheduler. Thus blocking can lead to extra overhead due to context switching or non-deterministic execution time due to scheduling variations. On the other hand, busy-waiting can waste a lot of processor cycles polling for lock acquisition, affecting both performance and energy consumption.

In some cases hardware can provide alternate mechanisms that allow the programmer to avoid explicit synchronization altogether. This is typically accomplished by exploiting either *decoupling* or *speculation*. Decoupling techniques remove the need to acquire locks by moving or buffering data in hardware. This approach requires no explicit synchronization between processes that wish to access shared data.

Examples of decoupling techniques include *hardware queues*, *direct memory access* (DMA), and *decorated storage operations*. Each of these provides a way for a producer of work or data to proceed independently of its consumer, thus avoiding explicit locking.

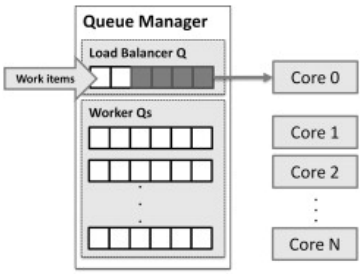
Speculation techniques assume a low probability of interference between processes sharing data. Under this assumption, execution can aggressively proceed. Speculation is complemented with the ability to defer committing any changes to architectural state until it is certain that there was in fact no interference between processes attempting to access shared data. Hardware support for speculation has been a research topic until very recently. The primary technique that has emerged as a candidate for inclusion in commercial multicore chips is *hardware transactional memory* (HTM) [9].

Messaging is a special case of decoupling that, when used, can pervade the programming model visible to the application programmer. A message between processes can achieve both communication and synchronization in a single action. While a message-based programming model can be implemented purely in software using locks and shared memory, some multicore systems are beginning to provide hardware-level support for messaging which can be used to efficiently implement lock-free programming.

The following sections will take a deeper look at various lock-free mechanisms provided by hardware, and the associated software considerations.

Lock-free synchronization with hardware queues

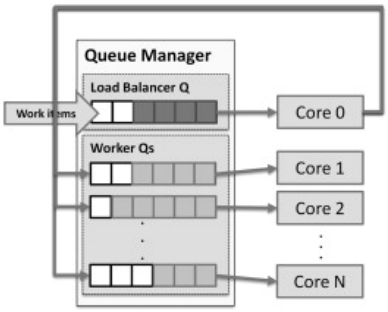
One decoupling approach to lock-free synchronization is the use of hardware queues. An example of this is the Freescale P4080 device, which has a highly configurable hardware queue manager [10]. A common application for this type of hardware queue in multicore systems is load balancing. In this scenario, one core can be assigned the task of receiving work items and then distributing them evenly across multiple worker cores for processing. Systems with these capabilities will often include autonomous I/O devices that can enqueue and dequeue items to the queue manager as well. Figure 14.14 depicts work arriving from an autonomous I/O device such as an Ethernet controller. The controller places items in the queue assigned to the load balancer core (Core 0 in the figure).



Download: Download full-size image

Figure 14.14. Work items enqueued for load balancer task.

Core 0 will evenly spread the work across remaining cores by enqueuing work items into queues assigned to each worker core (Figure 14.15); the worker cores will iteratively dequeue and process work items. In such systems an additional queue will typically be assigned for outgoing I/O as well. In the case of network packet forwarding software, for example, a worker core may finish a work item by enqueuing it to a hardware queue assigned to an outgoing Ethernet port, which would autonomously process egress of the packet. With this approach, locking can be completely avoided because each work item will be worked on by only one core at a time, eliminating context switches that might threaten thread safety.



Download: Download full-size image

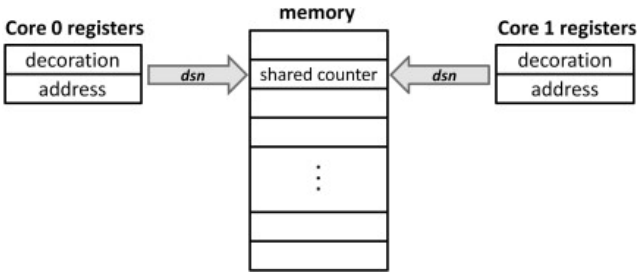
Figure 14.15. Work items enqueued for worker tasks.

Decorated storage operations

There are many situations in multicore programming when it is desirable to maintain shared data that serves as a counter or statistic of interest to the application. The nominal course of action to achieve this would be to use a lock variable to gain access to the counter value stored in shared memory and to update it within a critical section. This approach will certainly work, but in many situations it can have significant performance implications. Negative performance impacts are especially undesirable when the act of maintaining a counter is secondary to a requirement for achieving maximum processing throughput. An example of this situation is network packet-processing applications, wherein the main focus of the application is to route packets as fast as possible, and of secondary concern is a requirement to maintain statistics concerning the packets being routed.

Some multicore systems provide the means to maintain such counters using lock-free approaches. This is done via decorated storage operations, as in the Power Architecture `l*dx` (*decorated load family*), `st*dx` (*decorated store family*), and `dsn` (*decorated storage notify*) instructions. These instructions are uniformly defined from the processor point of view, but the semantics vary depending on the target device. For example, all decorated storage operations require two register arguments: register `rA` specifies the decoration semantics, register `rB` specifies the address. In addition, decorated store operations require a third argument: register `rS`, which contains additional data to be applied with the decoration. The decoration operation semantics will be defined according to the device targeted by the decorated storage operation. Examples include a decoration that increments the contents of a memory address and a decoration that adds a fixed value (perhaps contained in `rS`) to the contents of a memory address.

Figure 14.16 depicts the scenario of incrementing a shared counter using a `dsn` instruction. Each processor wishing to increment the counter will load the counter's address into `rB`, and an *increment location* command into `rA` (where *increment location* semantics and format are defined by the memory controller in use in the particular multicore system). When the processor executes the `dsn` instruction, a transaction will be sent to the memory controller (or perhaps a cache controller that will eventually flush the data to main memory), which will then atomically increment the counter without locking. This occurs atomically because the target device serializes all transactions that it receives. Note that for the increment scenario, it is not important in which order the increment updates occur, since, after the two operations complete, the counter will be correctly incremented by two.

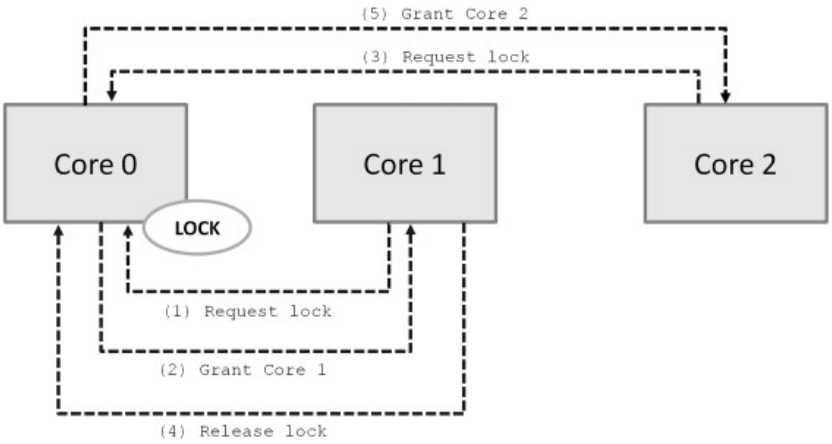


Download: Download full-size image

Figure 14.16. Decorated storage operations.

Messaging

Figure 14.17 illustrates how to achieve mutual exclusion synchronization using message passing. In this example, Core 0 owns a lock variable. Core 1 requests a lock via a message to Core 0 and is granted access to the lock via a message back from Core 0. During the time that Core 1 owns the lock, Core 2 also sends a request for the lock to Core 0. However, Core 0 does not grant Core 2 access to the lock until after it receives a message from Core 1 releasing the lock. Messaging can be used in similar fashion to implement various other synchronization models such as producer/consumer (e.g., software pipelines), barriers, and advanced synchronization techniques such as load balancing.



Download: Download full-size image

Figure 14.17. Implementing mutual exclusion synchronization with messaging.

In general, multicore systems can provide messaging support via hardware queues, DMA engines, or by inter-processor interrupt facilities. The special case of multicore systems without coherent memory support must provide messaging using a combination of hardware features. Each of these is described in more detail in the following sections.

Inter-device messaging with hardware queues

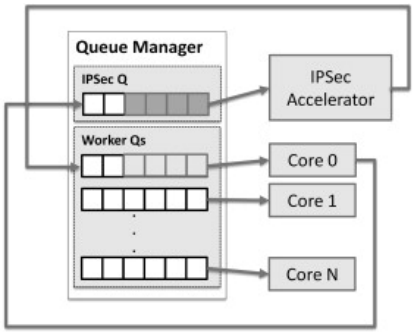
Many multicore chips also include various devices that offload processor cores from processing certain well-defined specific protocols. Operating system drivers for these protocols are often implemented purely as code to be run on the processor core, or may utilize the offload processor but remain inefficient due to high synchronization overheads required to share the resource amongst cores. For systems that offer specialized accelerators, using these drivers unmodified can mean that performance will be left on the table while the accelerator resources are underutilized.

One example is the IPsec protocol, which is used to encrypt Internet traffic for secure transactions between a web browser and a server. IPsec is an essential and frequently used protocol in the Internet. Thus, mainstream operating systems will provide generic IPsec drivers. However, because of the high processing demands of encryption and decryption, many multicore chips provide dedicated acceleration hardware for IPsec. As described above, a customized IPsec driver is required in order to take advantage of this hardware feature.

Even with customized drivers, performance issues can arise when multiple processors are attempting to share a dedicated hardware acceleration engine. In general, the interaction with an accelerator is to exchange messages between the processor and the acceleration engine. These messages often take the form of job descriptors. For IPsec processing, these job descriptors will likely contain information such as a pointer to the buffer of data to be processed, the particular security algorithm to be applied, and a key. The hardware accelerator reads a message from software, processes the data as requested, and then passes a message back to software. The flow of messages between cores and accelerators is typically managed through some form of first-in/first-out buffers (FIFO) queue. If this queue is implemented purely in shared memory and managed by software, then processors must use locks to enqueue and dequeue messages. This synchronization overhead results in lower throughput and poor utilization.

Devices such as the Freescale P2020 address this issue by providing a number of hardware queues for descriptors [11]. Queues are assignable to cores or to hardware accelerators, and descriptors can flow lock-free through these queues (Figure 14.18).

Experiments with off-the-shelf IPsec drivers modified to use hardware queues show that avoiding locks altogether can provide approximately 25%–30% throughput improvement.



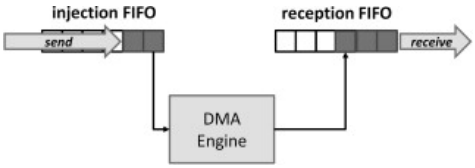
Download: Download full-size image

Figure 14.18. Messaging using hardware queues.

Messaging with DMA

It is often desirable to offload the processor core from the overhead of message delivery. This can be done by utilizing dedicated hardware which is first programmed by the processor core and then processes the message autonomously. A common approach is to leverage DMA devices.

Examples of this approach are the IBM Blue Gene/P and Blue Gene/Q systems [12], [13]. These systems include a DMA engine which can be dedicated to messaging. The provisioning of this DMA engine includes a collection of FIFOs for message injection and message reception. Figure 14.19 provides a high-level view of this approach. To perform a send operation, a message descriptor is placed in the injection FIFO. When the DMA engine processes the message descriptor, it will eventually result in an entry being appended to the appropriate reception FIFO. This entry will ultimately be read by software to perform the message receive.



Download: Download full-size image

Figure 14.19. Use of DMA for messaging.

The performance advantage of the DMA approach can be significant for large message sizes or for large numbers of messages. This allows the processor load for moving message descriptors to be quite small relative to the load on the DMA engine for moving entire messages. In addition, the FIFOs provide efficient decoupling of send and receive operations, allowing processors to avoid busy-waiting during send and receive operations.

The Blue Gene DMA hardware is complemented by the Deep Computing Messaging Interface (DCMF). DCMF provides a software interface to the messaging hardware, including support for *active messages*. Active messages differ from messaging models which require explicit receive actions to be issued by the receiver. Instead, active messages carry additional information in their header which identifies a software routine to be invoked on arrival of the header packet at its destination.

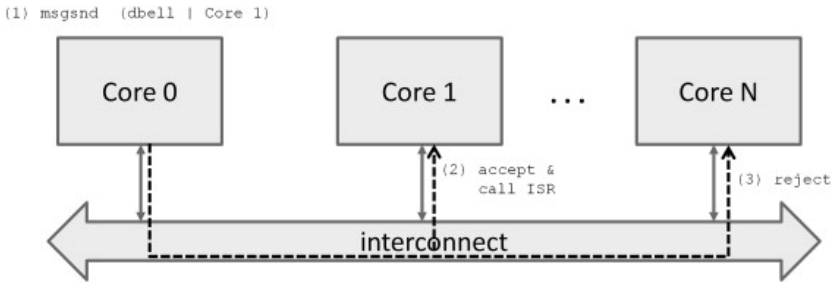
Inter-processor interrupt facilities

Some architectures include instruction-set support for messaging between processor cores via direct interrupt delivery. This approach to messaging completely bypasses the memory subsystem, thus requiring no locks. One example of this approach is provided by the Power Architecture `msgsnd` instruction. Processors initiate a message by executing the `msgsnd` instruction with a message type and message payload specified by the contents of register `rB`. Register `rB` is either a 32-bit or 64-bit register, so the payload is limited to a few bits of information indicating things such as a logical process ID or the specific processor ID for which

the doorbell is intended. Despite limited payload size, a mechanism like this makes it is easy and efficient to implement a messaging strategy supporting arbitrarily large messages. To achieve this, a message is first copied into shared memory by the sender and then a doorbell is sent to the intended target indicating that the message buffer is ready, and the receiver can subsequently read the message from shared memory. This provides a lock-free message send/receive transaction as long as the sender and receiver can agree not to access the shared memory buffer out of turn.

Executing the `msgsnd` instruction causes the hardware to broadcast the message to all processors within a coherence domain in a reliable manner. The message is transported over the system interconnect using a unique interconnect transaction type that can only be generated by the `msgsnd` instruction. This technique can work with any type of interconnect as long as the required attribute bits for the transaction type can be supported in transactions between master devices.

When a message is received by a processor, it is filtered in hardware based on the message type and payload. If the processor determines that it is a target for the message, an asynchronous interrupt service routine will be invoked to process the message (Figure 14.20).



Download: Download full-size image

Figure 14.20. Message send via doorbell interrupts.

Inter-core interrupt facilities are especially useful for implementing intra-operating system synchronization, as well as synchronization between a hypervisor and a guest operating system. To complete the hardware facilities for such interactions, the `msgsnd` instruction is complemented by interrupt enable/disable bits as well as a `msgclr` instruction that will clear any pending interrupt bits. These features allow the operating system or hypervisor software to fully manage the messaging facility to meet their demands.

Messaging in non-coherent systems

Anticipating an eventual move to manycore architectures, researchers at Intel have produced two generations of manycore chips under their "terascale" research program. The second-generation chip is called the Single Chip Cloud (SCC) [14], [15]. The SCC provides no cache-coherent shared address space, so coherency must be completely software managed.

The SCC chip consists of 48 P5 cores arranged in pairs on 24 tiles connected by a 2D mesh interconnect. Each core has access to private memory, and each tile provides 16KB of shared (but not coherent) memory known as the message-passing buffer (MPB). MPB memory is cacheable in the processor's private L1 cache, but bypasses the L2 cache. When data from the MPB is cached in the L1, the corresponding cache line is tagged as MPB data, enabling the memory subsystem to distinguish messages from other application data.

The normal Intel atomic instructions are not available in the SCC due to its lack of hardware coherency. Therefore, to provide a minimal locking capability, each processor has a `test_and_set` register which can be updated atomically by the hardware. In addition, the processor includes a new instruction (`cl1invmb`) to aid message passing. This single-cycle instruction is used to invalidate all L1 cache lines that are tagged as MPB data, causing subsequent memory accesses to go to backing store.

The SCC architecture defines conventions for interacting with the `test_and_set` registers, including specific data values used to indicate whether a lock has been set or cleared. Any SCC core can read any `test_and_set` register in the system, and this must be carefully managed in software. A core attempts to acquire a lock by reading a `test_and_set` register. The returned data value indicates whether the lock was obtained. Any core can reset a lock associated with any `test_and_set` register by writing the appropriate data value to the register. This capability provides a simple (but not highly scalable) capability to synchronize access

to the data associated with MPB for the purposes of messaging. All other synchronization in the system is built upon this capability using the message approach depicted in Figure 14.17.

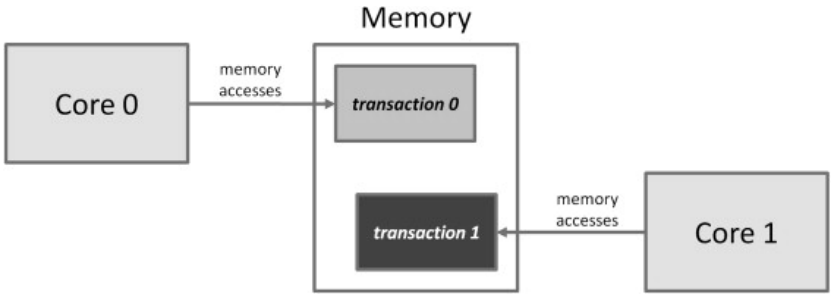
Given the basic synchronization mechanisms described above, messaging is then implemented in the SCC as follows. When a message is sent, the data is moved from core private memory through the L1 cache of the sending core. It is then moved to the MPB, and eventually to the L1 cache of the receiving core. The receiving process then reads the message from its L1 cache. The software responsible for negotiating the message exchange process stores various flags and other data in cache lines shared between the sending core and the receiving core (not to be confused with the MPB flags which are automatically handled by hardware). Access to the data in the shared cache line is guarded using locks managed in the `test_and_set` registers.

The SCC chip is a research vehicle, never intended for mass production. The lessons learned from this purely message-passing programming model should eventually materialize in production processors. Most SCC-based research has focused on how to make this programming model easily comprehensible by programmers while exhibiting good performance. It's important to remember when envisioning such future systems that performance will likely be evaluated using a throughput metric rather than a latency metric. We can expect such systems to run at lower frequencies for power efficiency reasons, and the combination of many processor cores on die and lower operating frequencies will affect how we think about programming and which algorithms we ultimately choose!

Hardware transactional memory

Another form of lock-free programming is hardware transactional memory (HTM). This approach is based on concepts from transactional databases. Much like concurrent programs interacting through shared memory, transactional databases allow concurrent accesses to data stored in their data files. Concurrent database transactions first snapshot the system state, then speculatively read and write data while logging the updates to non-permanent backing store, and finally commit the changes to permanent storage as long as no conflicting read or write accesses occurred during the transaction. If conflicts are detected, the permanent state is not changed and the commit fails (e.g., the transaction is *rolled back*).

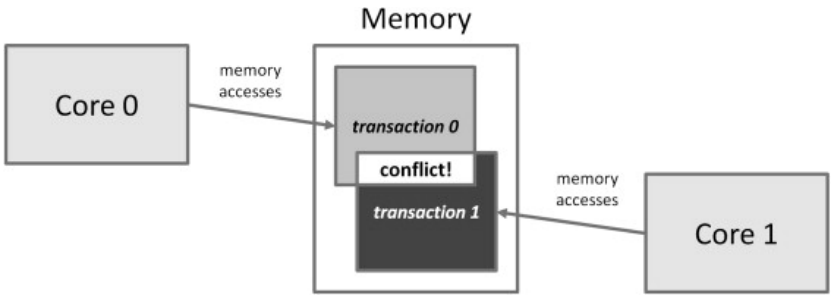
With HTM, a transaction is defined as a block of instructions containing a non-empty set of memory accesses. There can be no memory-level parallelism within a transaction, but separate transactions can proceed concurrently. Furthermore, a transaction is guaranteed to execute as an atomic unit if no memory access conflicts are detected (Figure 14.21). If a conflict is detected, then all instructions in the group will fail to execute. Conflicts occur when accesses for one transaction causally depend on accesses from another concurrent transaction. In a typical multicore system, the coherency mechanism will take care of broadcasting all changes related to a commit.



Download: Download full-size image

Figure 14.21. Concurrent memory transactions with no conflicts.

When a conflict is detected amongst concurrent transactions then the attempt to commit an associated transaction will fail and system state must reflect the state it was in before speculative execution of the transaction began (Figure 14.22). This requires hardware to provide the equivalent of the transactional database checkpointing and rollback features. Thus, with HTM the hardware is responsible for determining the set of memory accesses associated with a transaction and for providing conflict detection. This can be done, for example, by modifying the cache and the coherency protocol. In addition, the complexities of managing exceptions and interrupts are likely to be mitigated by simply aborting transactions in flight when these events occur.



Download: Download full-size image

Figure 14.22. Concurrent memory transactions with conflicts.

HTM is likely to become widely available over the next few years. One of the first commercial implementations to be announced is the IBM Blue Gene/Q system [16]. This system will employ a multi-versioning cache to support the transactions. Data in the cache will be tagged with additional bits allowing multiple versions of a memory location to be tracked using a scoreboarding approach. The scoreboard will be used to detect any load/store conflicts for a transaction. When a conflict is detected the system software will be notified and it will manage rollback and re-execution of the transaction.

The advantage of HTM is that programmers do not need to employ explicit synchronization. This removes significant control complexity and overhead from software implementation. However, the programmer must carefully determine transaction boundaries. When transactions frequently conflict there will be performance degradation, thus transaction granularity and degree of concurrency are important considerations. Similarly, large transactions will require the hardware to temporarily store more speculative state during transaction execution and subsequently broadcast more updates on a successful commit. Programmers must also be cognizant that it is not feasible to support arbitrarily large or arbitrarily many transactions purely in hardware due to the limited amount of resources available to store speculative state for open transactions.

Memory subsystem considerations

Most of the synchronization mechanisms presented in this chapter rely on shared memory and associated coherency of data. For example, ensuring atomicity of a load-reserved/store-conditional instruction sequence depends on the ability of the hardware to detect whether or not an intervening store has occurred to the reserved address (thus violating the reservation). The ideal guarantee that programmers would like from the hardware is *sequential consistency*, as defined below [17]:
A system is sequentially consistent if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in the order specified by the program

Leslie Lamport

However, for a variety of performance reasons, multicore processor architectures provide a range of memory consistency semantics. For example, caches can sometimes prevent the effect of a store from being seen by other processors. These multicore complications motivate architects to provide weak or relaxed memory consistency models. Unfortunately, this comes at a price for programmers, who must be aware of these relaxations in order for loads and stores to ensure that their code is functionally correct.

Memory ordering rules

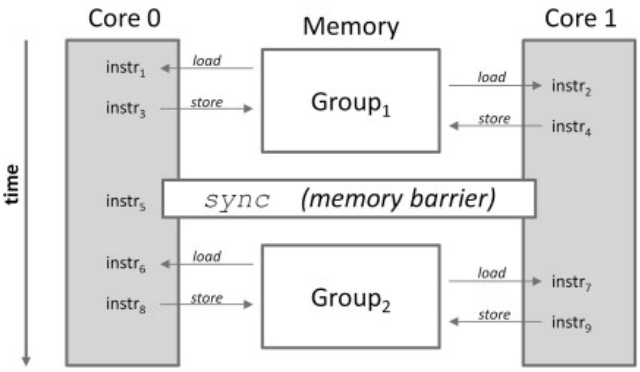
Strongly consistent memory ordering rules require that all processes sharing data in a multicore system will observe all updates to the shared data in the same order (typically the exact order in which they occur). In contrast, weakly consistent ordering rules allow the order as performed by processors to differ from the order in which updates occur at main memory. Thus different processors may observe different update orders. In many cases, this poses no significant problem since the majority of memory accesses are likely to be non-conflicting.

The biggest effect of memory access reordering is manifested in operating system routines, libraries for synchronization, and device drivers. Fortunately, these are relatively well-contained and highly reused areas of code, and the performance benefits justify the additional programming and verification complexity. For completeness, architectures with relaxed memory order

must also provide memory fence instructions to allow programmers to enforce serialization of memory accesses when it is essential for program correctness. Weakly ordered architectures include SPARC (`stbar` and `membar` instructions), newer Intel architectures (`sfence`, `lfence`, and `mfence` instructions), and Power Architecture (`sync`, `mbar`, `lwsync` instructions).

The `sfence` instruction guarantees that all store instructions executed prior to its execution will complete before the processor will execute any additional instructions. Similarly, the `lfence` instruction ensures that all load instructions executed prior to its execution will complete before proceeding. The `mfence` instruction combines the effects of `sfence` and `lfence`, ensuring that both loads and store executed before the `mfence` will complete before proceeding.

Power Architecture provides a rich set of options for the programmer to manage the effects of weakly consistent ordering rules. The `sync` instruction (also known as `sync 0` or `msync`) ensures that all memory accesses executed before the `sync` instruction ($Group_1$ in Figure 14.23) will be completed before any accesses following the `sync` instruction ($Group_2$ in Figure 14.23), and, in fact, ensures that no instructions following the `sync` start their execution until the `sync` completes. The `mbar` instruction (also known as `mbar 0`) is similar to `sync`, but only affects load and store instructions. The `mbar 1` variation achieves better performance by relaxing completion rules to cover only (i) stores that are to locations in write-through cache, (ii) loads and stores that are both cache-inhibited and guarded (as in data for device drivers), and (iii) stores to normal cacheable coherent memory. The `lwsync` instruction (also known as `sync 1`) performs a lightweight operation that will complete normal cacheable memory accesses but will not order a store followed by a load.



Download: Download full-size image

Figure 14.23. Effects of `sync` instruction.

For certain applications, it is more convenient to designate behavior for a range of data rather than issuing synchronizing instructions within the code. This can generally be provided via configuration options at the system level or by designating attributes for physical pages in memory (in the memory management unit or elsewhere). For example, Power Architecture provides for a bit in a processor configuration register to specify at the system level that all cache-inhibited and guarded loads and stores shall be performed in order (`HID0[CIGLS0]` bit). And certain Intel architectures support memory type range registers (`MTRR`) that can be used to define memory ordering rules for specific regions of memory, while other Intel architectures provide page attribute tables (`PAT`) that can be used to strengthen memory ordering rules for specific memory pages.

Using memory barriers and synchronizations

Proper management of memory consistency only adds a few lines of code to important routines, but knowing when and how to use memory barriers is a very important consideration. In general, whenever an atomic operation will depend on coherency of the data then the programmer will need to insert some form of barrier into their code.

In the example Power Architecture code shown in Figure 14.24, a shared memory location is used as a lock to control access to a shared resource (such as an I/O device). The lock is available when its value is 0, and locked when its value is 1. This example code uses the `test_and_set()` routine from Figure 14.10 to obtain the lock. The lock routine contains a `sync` instruction since reads to the shared resource cannot be executed until the lock has been acquired. Because of the `sync` instruction, any speculative reads to the shared resource that have been issued by the processor will be discarded (such reads may have occurred, for example, due to a misprediction by the branch predictor).

```
resource-lock
lock:      mfspr   r6, LR        # save Link Register
           addi    r4, r0, 1     # obtain lock:
loop:      bl      test_and_set
           bc      4, 2, loop    # retry until lock obtained
# Delay instructions below until instructions above complete
           sync    0             # or isync, mbar, lwsync, …
           mtspr   LR, r6        # restore link register
           blr                   # return
```

Download: Download full-size image

Figure 14.24. Assembly code for resource lock.

A complementary resource-unlocking procedure must be created. This procedure will store a 0 to the lock location. In this case, the programmer will want to ensure that all stores to the shared resource complete before the lock is released (some of the stores may be buffered by the system for deferred write, for example). In the example shown in Figure 14.25 the resource-unlock procedure begins with a `sync` barrier to ensure this property.

```
resource-unlock
unlock:    sync    0             # or isync, mbar, lwsync, …
# Delay instructions below until instructions above complete
           addi    r1, r0, 0     # prepare value
           stw     r1, r0, 0     # store 0 to lock location
           blr                   # return
```

Download: Download full-size image

Figure 14.25. Assembly code for resource unlock.

In addition to functional correctness, the programmer must also be aware of the performance implications of managing weakly consistent memory. For example, to help guarantee forward progress in routines such as the resource lock/unlock described above, it is important to minimize looping on load-reserved/store-conditional transaction pairs. Code should be carefully examined to ensure that actions which would cause loss of the reservation are minimized; such actions cause more time to be spent in the lock acquisition loop than is strictly necessary. In other cases, there may be a benefit to performing a regular load instruction to test a value before performing the load-reserved transaction (e.g., `lwarx`). Depending on the semantics of the target resource and the type of accesses made to it, the choice of which barrier instruction to use could range from the most heavyweight form (e.g., `sync`) to the most lightweight form (e.g., `lwsync`). The programmer must trade off correctness with performance when making this choice. These considerations are highly processor-dependent, so programmers should carefully examine their code with respect to the semantics of their system.

Conclusions

Synchronization is an essential element of multicore programming, and it comes in many forms. Underlying hardware must provide support for synchronization, and does so in the form of a few primitive atomic memory read/modify operations upon which many synchronization idioms can be built. For performance and complexity reasons, atomic synchronization primitives do not scale well as chips move to more and more cores. Therefore, instruction sets also provide various speculation mechanisms which can improve performance. These include paired memory read/modify instructions, which provide speculative atomicity with violation detection, and weakly ordered memory semantics, which allow loads and stores to be performed out of order. In addition, based on the recognition that synchronization can remain a bottleneck even with the most efficient underlying hardware support, several lock-free mechanisms have emerged that can further improve software performance by avoiding synchronization altogether.

Recommended articles

References

[1] G.L. Peterson
Myths about the mutual exclusion problem
Inform. Process. Lett., 12 (3) (1981), pp. 115-116
View PDF View article View in Scopus ↗ Google Scholar ↗

[2] D.L. Weaver, T. Germond, The SPARC Architecture Manual. 1994; Available from: <www.sparc.org/standards/SPARCV9.pdf ↗>.
Google Scholar ↗

[3] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer Manuals 2011 [cited]; Available from: <http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html ↗>.
Google Scholar ↗

[4] A. Sood, Guide to Porting from Solaris to Linux on x86. 2005; Available from: <https://www.ibm.com/developerworks/linux/library/l-solar/ ↗>.
Google Scholar ↗

[5] Freescale. EREF 2.0: A Programmer's Reference Manual for Freescale Power Architecture Processors. 2011; Available from: <http://cache.freescale.com/files/32bit/doc/ref_manual/EREF_RM.pdf ↗>.
Google Scholar ↗

[6] D. Chun, S. Latif, MIPS R4000 Synchronization Primitives. 1993; Available from: <http://www.mips.com/products/product-materials/processor/archives/ ↗>.
Google Scholar ↗

[7] A.A. Committee
Alpha Architecture Reference Manual (third ed.), Elsevier (1998)
952
Google Scholar ↗

[8] The Free Software Foundation. GCC 4.6.2 Manual. 2011; Available from: <http://gcc.gnu.org/onlinedocs/gcc-4.6.2/gcc/ ↗>.
Google Scholar ↗

[9] S. Liu, J.-L. Gaudiot, Synchronization mechanisms on modern multi-core architectures, in: Twelfth Asia-Pacific Computer Systems Architecture Conference (ACSAC 2007). 2007.
Google Scholar ↗

[10] Freescale Semiconductor, I. P4080 product summary page. 2008 [cited; Available from: <www.freescale.com/files/netcomm/doc/fact_sheet/QorIQ_P4080.pdf ↗>.
Google Scholar ↗

[11] S. Malik, R. Malhotra, Performance enhancement for ipsec processing on multi-core systems. In: 2011 International Conference on Communications, 2011.
Google Scholar ↗

[12] D. Chen, et al. The IBM Blue Gene/Q interconnection network and message unit. In Supercomputing 2011 (SC11), 2011.
Google Scholar ↗

[13] S. Kumar, et al. The deep computing messaging framework: generalized scalable message passing on blue gene/p supercomputer. In: twenty second Annual International COnference on Supercomputing, 2008.
Google Scholar ↗

[14] Intel Corporation. SCC External Architecture Specification. 2010; Available from: <http://communities.intel.com/docs/DOC-5852 ↗>.

Google Scholar ↗

[15] R.F. van der Wijngaart, T.G. Mattson, W. Haas
Light-weight communications on Intel's single-chip cloud computer processor
ACM SIGOPS Operating Systems Review, 45 (1) (2011), pp. 73-83

Crossref ↗ Google Scholar ↗

[16] IBM. IBM system blue Gene/Q data sheet. 2011; Available from: <http://www-03.ibm.com/systems/deepcomputing/solutions/bluegene/ ↗>.

Google Scholar ↗

[17] L. Lamport
How to make a multiprocessor computer that correctly executes multiprocess programs
IEEE Trans. Comput., 28 (9) (1979), pp. 690-691

View in Scopus ↗ Google Scholar ↗

Cited by (0)

View Abstract

Copyright © 2013 Elsevier Inc. All rights reserved.

ELSEVIER

All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

RELX™